



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное
учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА КОМПЬЮТЕРНЫЕ СИСТЕМЫ И СЕТИ (ИУ6)

НАПРАВЛЕНИЕ ПОДГОТОВКИ 09.03.01 Информатика и вычислительная техника

ОТЧЕТ ПО УЧЕБНОЙ ПРАКТИКЕ

Студент Соболев Сергей Михайлович
Фамилия имя отчество

Группа ИУ6-23Б

Тип практики Проектно-технологическая практика

Название
предприятия НУК ИУ МГТУ им. Н.Э. Баумана

Студент С. М. Соболев
(Подпись, дата) (И.О. Фамилия)

Руководитель
практики О. А. Веселовская
(Подпись, дата) (И.О. Фамилия)

Оценка отлично (100 из 100)

2026 г.

ЗАДАНИЕ на учебную практику

по теме Проектирование и реализация программного обеспечения с использованием
структурного и объектного подходов

Студент группы ИУ6-23Б

Соболев Сергей Михайлович
(Фамилия, имя, отчество)

Тип практики Проектно-технологическая практика

Название предприятия НУК ИУ МГТУ им. Н.Э. Баумана

Техническое задание:

Задание 1. Создание информационной программной системы с графическим интерфейсом на C++

Выполнить объектную декомпозицию, разработать формы интерфейса, диаграмму состояний интерфейса, диаграммы классов интерфейсной и предметной областей, диаграмму последовательности действий одной из реализуемых операций. Разработать, протестировать и отладить программу в среде Visual Studio или QT Creator.

База данных (файл) продуктового магазина содержит сведения о товарах: номер партии, наименование товара, количество (штук), дата поставки, срок годности (в днях). Программа должна в интерактивном режиме формировать файл, добавлять и удалять данные, а также воспринимать каждый из перечисленных запросов и давать на него ответ.

1. Показать все сведения о товарах, поступивших с указанной даты.
2. Определить наименования товаров, срок годности которых истекает сегодня.
3. Определить самый скоропортящийся товар (если их несколько, показать все).
4. Построить график изменения объема поступлений заданного товара по датам.

Задание 3 Изучение средств создания графических интерфейсов на C#

Задание выполняется по методическим указаниям к заданию 3.

Задание 4 Создание программной системы с графическим интерфейсом на C#

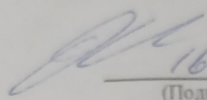
Выполнить объектную декомпозицию, разработать формы интерфейса, диаграмму состояний интерфейса, диаграммы классов интерфейсной и предметной областей, диаграмму последовательности действий одной из реализуемых операций. Разработать, протестировать и отладить программу.

База данных (файл) продуктового магазина содержит сведения о товарах: номер партии, наименование товара, количество (штук), дата поставки, срок годности (в днях). Программа должна в интерактивном режиме формировать файл, добавлять и удалять данные, а также воспринимать каждый из перечисленных запросов и давать на него ответ.

1. Показать все сведения о товарах, поступивших с указанной даты.
2. Определить наименования товаров, срок годности которых истекает сегодня.
3. Определить самый скоропортящийся товар (если их несколько, показать все).

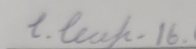
Дата выдачи задания « 07 » февраля 2025 г.

Руководитель практики

 16.02.2026
(Подпись, дата)

О.А. Веселовская
(И.О. Фамилия)

Студент

 16.02.2026
(Подпись, дата)

С. М. Соболев
(И.О. Фамилия)

Введение.....	4
1. Создание информационной программной системы с графическим интерфейсом на C++...	6
1.1. Объектная декомпозиция приложения.....	7
1.2. Разработка диаграмм классов и интерфейса.....	7
1.3. Разработка кода приложения.....	10
1.4. Тестирование приложения.....	18
2. Создание приложения с графическим интерфейсом на языке C# («Крестики-нолики»)...	21
2.1. Проектирование приложения.....	22
2.2. Разработка кода программы.....	22
2.3. Тестирование программы.....	24
3. Создание программной системы с графическим интерфейсом на языке C#.....	27
3.1. Проектирование приложения.....	28
3.2. Разработка кода программы.....	30
Заключение.....	35

Введение

Главная цель прохождения учебной практики заключается в приобретении практического опыта разработки компактных программных продуктов, оснащенных как графическим, так и консольным пользовательским интерфейсом.

Для достижения этой цели были поставлены следующие задачи:

- углубленное освоение инструментов и технологий создания программного обеспечения на современных языках программирования высокого уровня;
- изучение методологий и наработка опыта проектирования архитектуры небольших приложений с применением принципов структурного и объектно-ориентированного подходов;

Успешное выполнение практикума направлено на выработку и закрепление следующих компетенций:

- способность анализировать предметную область, выделять ключевые сущности, формировать на их основе классы и выстраивать логическое взаимодействие между ними;
- создание удобного и интуитивно понятного пользовательского взаимодействия (проектирование эргономики информационных систем);
- написание, тестирование и отладка программного кода с использованием актуальных интегрированных сред разработки и автоматизированных средств проектирования;
- составление проектной, технической и эксплуатационной документации для создаваемого программного обеспечения;

Задание 1. Создание информационной программной системы с графическим интерфейсом на C++

Задание: Выполнить объектную декомпозицию, разработать формы интерфейса, диаграмму состояний интерфейса, диаграммы классов интерфейсной и предметной областей, диаграмму последовательности действий одной из реализуемых операций. Разработать, протестировать и отладить программу в среде Visual Studio или QT Creator.

База данных (файл) продуктового магазина содержит сведения о товарах: номер партии, наименование товара, количество (штук), дата поставки, срок годности (в днях). Программа должна в интерактивном режиме формировать файл, добавлять и удалять данные, а также воспринимать каждый из перечисленных запросов и давать на него ответ.

1. Показать все сведения о товарах, поступивших с указанной даты.
2. Определить наименования товаров, срок годности которых истекает сегодня.
3. Определить самый скоропортящийся товар (если их несколько, показать все).
4. Построить график изменения объема поступлений заданного товара по датам.

1.1 Объектная декомпозиция приложения

На начальном этапе проектирования программного продукта была выполнена объектная декомпозиция, выделены основные модули приложения и их взаимодействия.

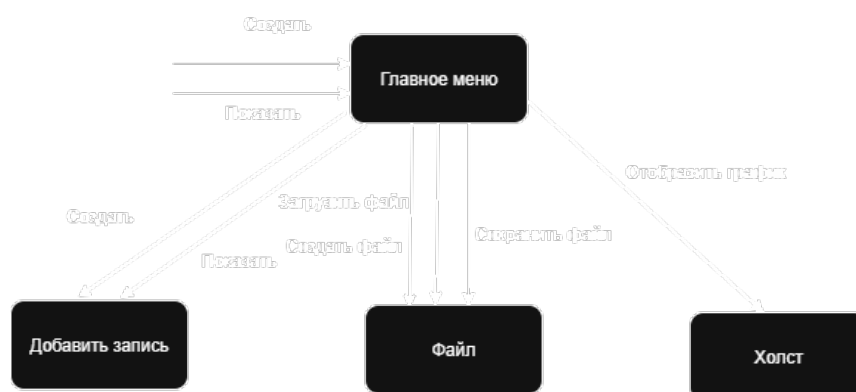


Рисунок 1 - Объектная декомпозиция

1.2 Разработка диаграмм классов и интерфейса

Для реализации графического интерфейса пользователя были спроектированы соответствующие окна, классы и структуры данных. Диаграммы классов интерфейсной области и последовательности действий представлены на рисунках ниже.

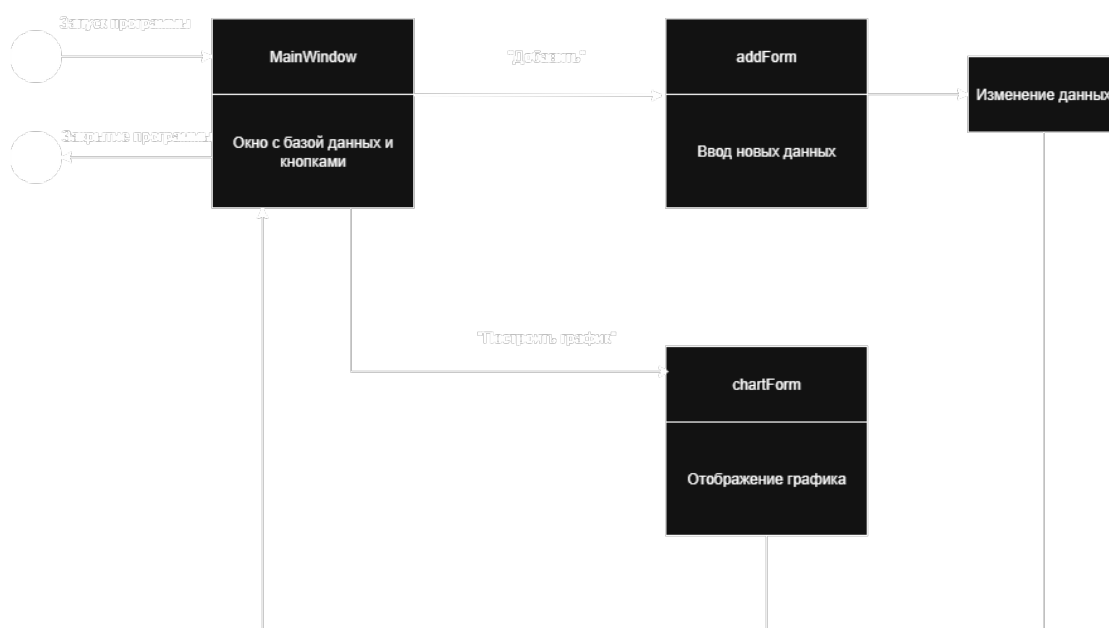


Рисунок 2 - Диаграмма классов интерфейса

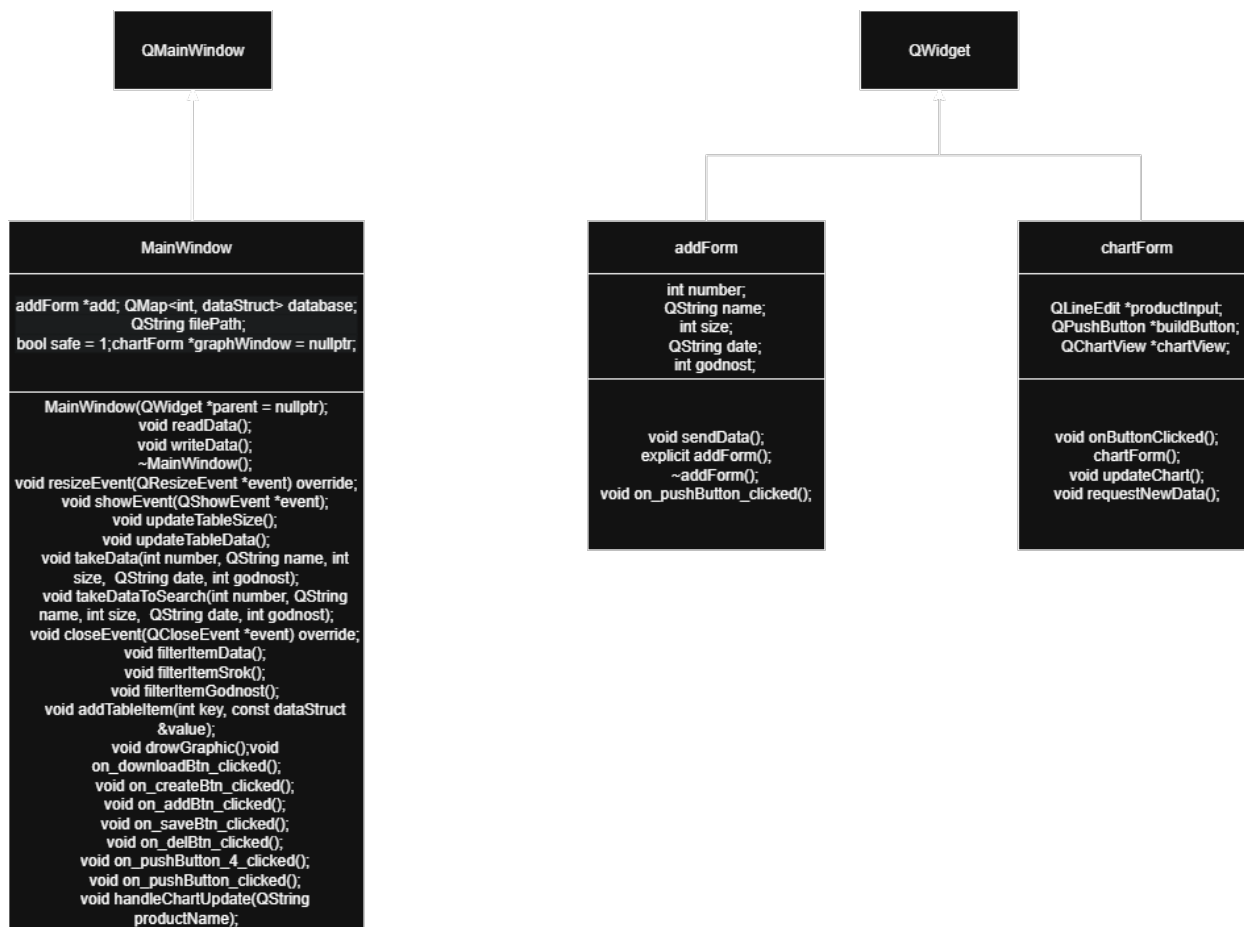


Рисунок 3 - Диаграмма классов интерфейсной области

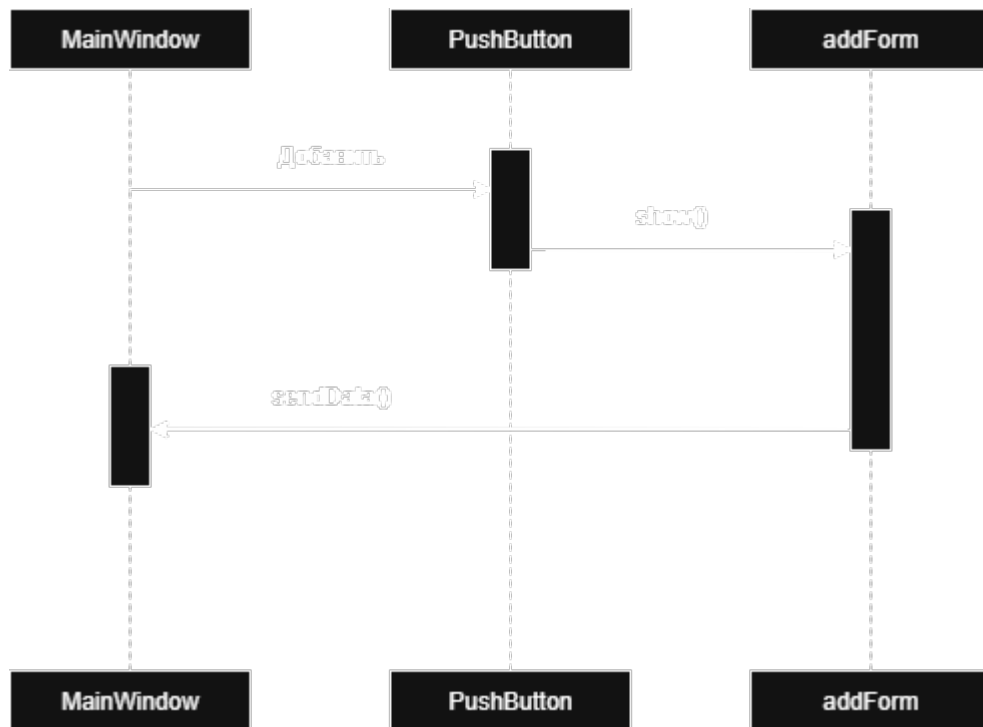


Рисунок 4 - Диаграмма последовательности действий

```

struct dataStruct {
    int number;
    QString name;
    int size;
    QString date;
    int godnost;
};
  
```

Рисунок 5 - Структура, отвечающая за данные

1.3 Разработка кода приложения

На основе спроектированных диаграмм был написан программный код приложения. Были реализованы функции чтения и записи файлов, обработки сигналов от кнопок, а также отрисовки таблиц и графиков.

```
class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    MainWindow(QWidget *parent = nullptr);
    void readData();
    void writeData();
    ~MainWindow();

private:
    Ui::MainWindow *ui;
    bool safe = 1;
    chartForm *graphWindow = nullptr;

protected:
    void resizeEvent(QResizeEvent *event) override;
    void showEvent(QShowEvent *event);
    void updateTableSize();
    addForm *add;
    QMap<int, dataStruct> database;
    QString filePath;
    void updateTableData();
    void takeData(int number, QString name, int size, QString date, int godnost);
    void takeDataToSearch(int number, QString name, int size, QString date, int godnost);
    void closeEvent(QCloseEvent *event) override;
    void filterItemData();
    void filterItemSrok();
    void filterItemGodnost();
    void addTableItem(int key, const dataStruct &value);
    void drawGraphic();

private slots:
    void on_downloadBtn_clicked();
    void on_createBtn_clicked();
    void on_addBtn_clicked();
    void on_saveBtn_clicked();
    void on_delBtn_clicked();
    void on_pushButton_4_clicked();
    void on_pushButton_clicked();
    void handleChartUpdate(QString productName);
};
```

Рисунок 6 - Заголовочный файл MainWindow

```

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
{
    ui->setupUi(this);
    add = new addForm();
    connect(add, &addForm::sendData, this, &MainWindow::takeData);
    ui->tableWidget->setEditTriggers(QAbstractItemView::NoEditTriggers);
    ui->tableWidget->setVerticalScrollBarPolicy(Qt::ScrollBarAlwaysOn);
}

MainWindow::~MainWindow()
{
    delete ui;
}

void MainWindow::updateTableSize(){
    int tableWidth = ui->tableWidget->viewport()->width() ;
    ui->tableWidget->horizontalHeader()->setSectionResizeMode(QHeaderView::Fixed);
    ui->tableWidget->setColumnWidth(0, tableWidth * 0.15);
    ui->tableWidget->setColumnWidth(1, tableWidth * 0.25);
    ui->tableWidget->setColumnWidth(2, tableWidth * 0.2);
    ui->tableWidget->setColumnWidth(3, tableWidth * 0.18);
    ui->tableWidget->setColumnWidth(4, tableWidth * 0.22);
}

void MainWindow::showEvent(QShowEvent *event) {
    QMainWindow::showEvent(event);
    updateTableSize();
}

void MainWindow::resizeEvent(QResizeEvent *event) {
    QMainWindow::resizeEvent(event);
    updateTableSize();
}

```

Рисунок 7 - Релизация отрисовки таблицы и конструктора

```

void MainWindow::readData(){
    QFile file(filePath);
    if (!file.open(QIODevice::ReadOnly)) qDebug() << "Ошибка";

    QTextStream out(&file);
    database.clear();
    while (!out.atEnd()) {
        QString line = out.readLine();
        QStringList parts = line.split(";");
        qDebug() << parts.size();
        if (parts.size() == 5) {
            int id = parts[0].toInt();
            dataStruct dataTemp;
            dataTemp.number = id;
            dataTemp.name = parts[1];
            dataTemp.size = parts[2].toInt();
            dataTemp.date = parts[3];
            dataTemp.godnost = parts[4].toInt();
            database.insert(id, dataTemp);
            qDebug() << "Запись";
        }
    }
    updateTableData();
    file.close();
}

void MainWindow::writeData(){
    QFile file(filePath);
    if (!file.open(QIODevice::WriteOnly)) qDebug() << "Ошибка";
    QTextStream out(&file);
    QMapIterator<int, dataStruct> i(database);
    while (i.hasNext()) {
        i.next();
        int id = i.key();
        dataStruct dataTemp = i.value();
        out << id << ";"
            << dataTemp.name << ";"
            << dataTemp.size << ";"
            << dataTemp.date << ";"
            << dataTemp.godnost << "\n";
    }
    file.close();
}

```

Рисунок 8 - Реализация чтения/записи файлов

```

void MainWindow::on_downloadBtn_clicked()
{
    filePath = QFileDialog::getOpenFileName(
        this,
        "Выберите файл для загрузки",
        "C:/Users/user/Documents/BMSTUC/1 course/sem2/Практика/ProdyctMagazine/data",
        "Текстовые файлы (*.txt);;Все файлы (*.*)"
    );
    if (filePath.isEmpty()) return;
    readData();
}

void MainWindow::on_createBtn_clicked()
{
    filePath = QFileDialog::getSaveFileName(
        this,
        "Создать и сохранить файл",
        "C:/Users/user/Documents/BMSTUC/1 course/sem2/Практика/ProdyctMagazine/data",
        "Текстовые файлы (*.txt);;Все файлы (*.*)"
    );

    if (filePath.isEmpty()) {
        return;
    }

    QFile newFile(filePath);
    if (newFile.open(QIODevice::WriteOnly | QIODevice::Text)) {
        newFile.close();
    }
}

```

Рисунок 9 - Реализация сигналов в MainWindow

```

void MainWindow::on_addBtn_clicked()
{
    if (filePath.isEmpty()) {QMessageBox::warning(this, "Внимание", "Файл не выбран.");
        return;}
    add->show();
}

void MainWindow::on_saveBtn_clicked()
{
    if (!filePath.isEmpty()){
        QMessageBox::information(this, "Информация", "Данные сохранены.");
        writeData();}
    else{
        QMessageBox::warning(this, "Внимание", "Файл не выбран.");
    }
    safe = 1;
}

void MainWindow::on_delBtn_clicked()
{
    int row = ui->tableWidget->currentRow();
    if (row < 0){
        QMessageBox::warning(this, "Внимание", "Строка не выбрана.");
        return;
    }
    QTableWidgetItem *item = ui->tableWidget->item(row, 0);
    int id = item->text().toInt();
    database.remove(id);
    updateTableData();
}

```

Рисунок 10 - Реализация сигналов в MainWindow

```

void MainWindow::addTableItem(int key, const dataStruct &value){
    int row = ui->tableWidget->rowCount();
    ui->tableWidget->insertRow(row);

    QTableWidgetItem *itemId = new QTableWidgetItem(QString::number(key));
    QTableWidgetItem *itemName = new QTableWidgetItem(value.name);
    QTableWidgetItem *itemSize = new QTableWidgetItem(QString::number(value.size));
    QTableWidgetItem *itemDate = new QTableWidgetItem(value.date);
    QTableWidgetItem *itemGodnost = new QTableWidgetItem(QString::number(value.godnost));

    ui->tableWidget->setItem(row, 0, itemId);
    ui->tableWidget->setItem(row, 1, itemName);
    ui->tableWidget->setItem(row, 2, itemSize);
    ui->tableWidget->setItem(row, 3, itemDate);
    ui->tableWidget->setItem(row, 4, itemGodnost);
}

```

Рисунок 11 – Функция, добавляющая предмет в таблицу

```

#ifndef ADDFORM_H
#define ADDFORM_H

#include <QWidget>

namespace Ui {
class addForm;
}

class addForm : public QWidget
{
    Q_OBJECT

public:
    explicit addForm(QWidget *parent = nullptr);
    ~addForm();

signals:
    void sendData(int number,
                  QString name,
                  int size,
                  QString date,
                  int godnost);

private slots:
    void on_pushButton_clicked();

private:
    Ui::addForm *ui;
};

#endif // ADDFORM_H

```

Рисунок 12 – Заголовочный файл addForm

```

#include "addform.h"
#include "ui_addform.h"
#include <QString>

addForm::addForm(QWidget *parent)
    : QWidget(parent)
    , ui(new Ui::addForm)
{
    ui->setupUi(this);
    ui->dateEdit->setDate(QDate::currentDate());
    ui->dateEdit->setCalendarPopup(true);
}

addForm::~addForm()
{
    delete ui;
}

void addForm::on_pushButton_clicked()
{
    int number = ui->lineEdit->text().toInt();
    QString name=ui->lineEdit_2->text();
    int size=ui->lineEdit_3->text().toInt();
    QString date=ui->dateEdit->text();
    int godnost=ui->lineEdit_6->text().toInt();

    emit sendData(number, name, size, date, godnost);

    ui->dateEdit->clear();
    ui->lineEdit->clear();
    ui->lineEdit_2->clear();
    ui->lineEdit_3->clear();
    ui->lineEdit_6->clear();
}

```

Рисунок 13 – Реализация addForm

```

#ifndef CHARTFORM_H
#define CHARTFORM_H

#include <QWidget>
#include <QtCharts/QChartView>
#include <QtCharts/QChart>
#include <QtCharts/QLineSeries>
#include <QtCharts/QDateTimeAxis>
#include <QtCharts/QValueAxis>
#include <QVBoxLayout>
#include <QHBoxLayout>
#include <QLineEdit>
#include <QPushButton>
#include <QMap>
#include <QDate>
#include <QString>

class chartForm : public QWidget {
    Q_OBJECT

public:
    chartForm(const QString &productName, const QMap<QDate, int> &chartData);

    void updateChart(const QString &productName, const QMap<QDate, int> &chartData);

signals:
    void requestNewData(QString productName);

private slots:
    void onButtonClicked();

private:
    QLineEdit *productInput;
    QPushButton *buildButton;
    QChartView *chartView;
};

#endif // CHARTFORM_H

```

Рисунок 14 – Заголовочный файл chartForm

```

#include "chartform.h"

chartForm::chartForm(const QString &productName, const QMap<QDate, int> &chartData) {
    // 1. Создаем строку ввода и кнопку
    productInput = new QLineEdit(productName); // Сразу вводим имя такого товара
    buildButton = new QPushButton("Показать график");
    connect(buildButton, &QPushButton::clicked, this, &chartForm::onButtonClicked);

    // 2. Создаем верхнюю горизонтальную панель и вставляем туда наши виджеты
    QHBoxLayout *topPanel = new QHBoxLayout();
    topPanel->addWidget(productInput);
    topPanel->addWidget(buildButton);

    // 3. Строим сам график (это такой готовый код)
    QChart *chart = new QChart();
    chart->legend()->hide();
    QLineSeries *series = new QLineSeries();

    QMapIterator<QDate, int> j(chartData);
    while (j.hasNext()) {
        j.next();
        qreal x = j.key().startOfDay().toMSecsSinceEpoch();
        qreal y = j.value();
        series->append(x, y);
    }

    chart->addSeries(series);
    chart->setTitle("Динамика поступлений: " + productName);

    QDateTimeAxis *axisX = new QDateTimeAxis();
    axisX->setFormat("dd.MM.yyyy");
    axisX->setTitleText("Дата поставки");
    axisX->setTickCount(5);
    chart->addAxis(axisX, Qt::AlignBottom);
    series->attachAxis(axisX);

    QValueAxis *axisY = new QValueAxis();
    axisY->setTitleText("Объем товара (шт)");
    axisY->setLabelFormat("%i");
    chart->addAxis(axisY, Qt::AlignLeft);
    series->attachAxis(axisY);

    chartView = new QChartView(chart);

    QVBoxLayout *mainLayout = new QVBoxLayout(this);
    mainLayout->addLayout(topPanel);
    mainLayout->addWidget(chartView);

    this->setWindowFlags(Qt::Window);
    this->resize(800, 800);
    this->setAttribute(Qt::WA_DeleteOnClose);
}

void chartForm::onButtonClicked() {
    QString newProduct = productInput->text();
    emit requestNewData(newProduct);
}

void chartForm::updateChart(const QString &productName, const QMap<QDate, int> &chartData) {
    QChart *newChart = new QChart();
    newChart->legend()->hide();

    QLineSeries *series = new QLineSeries();
    QMapIterator<QDate, int> j(chartData);
    while (j.hasNext()) {
        j.next();
        qreal x = j.key().startOfDay().toMSecsSinceEpoch();
        qreal y = j.value();
        series->append(x, y);
    }

    newChart->addSeries(series);
    newChart->setTitle("Динамика поступлений: " + productName);

    QDateTimeAxis *axisX = new QDateTimeAxis();
    axisX->setFormat("dd.MM.yyyy");
    axisX->setTitleText("Дата поставки");
    axisX->setTickCount(5);
    newChart->addAxis(axisX, Qt::AlignBottom);
    series->attachAxis(axisX);

    QValueAxis *axisY = new QValueAxis();
    axisY->setTitleText("Объем товара (шт)");
    axisY->setLabelFormat("%i");
    newChart->addAxis(axisY, Qt::AlignLeft);
    series->attachAxis(axisY);

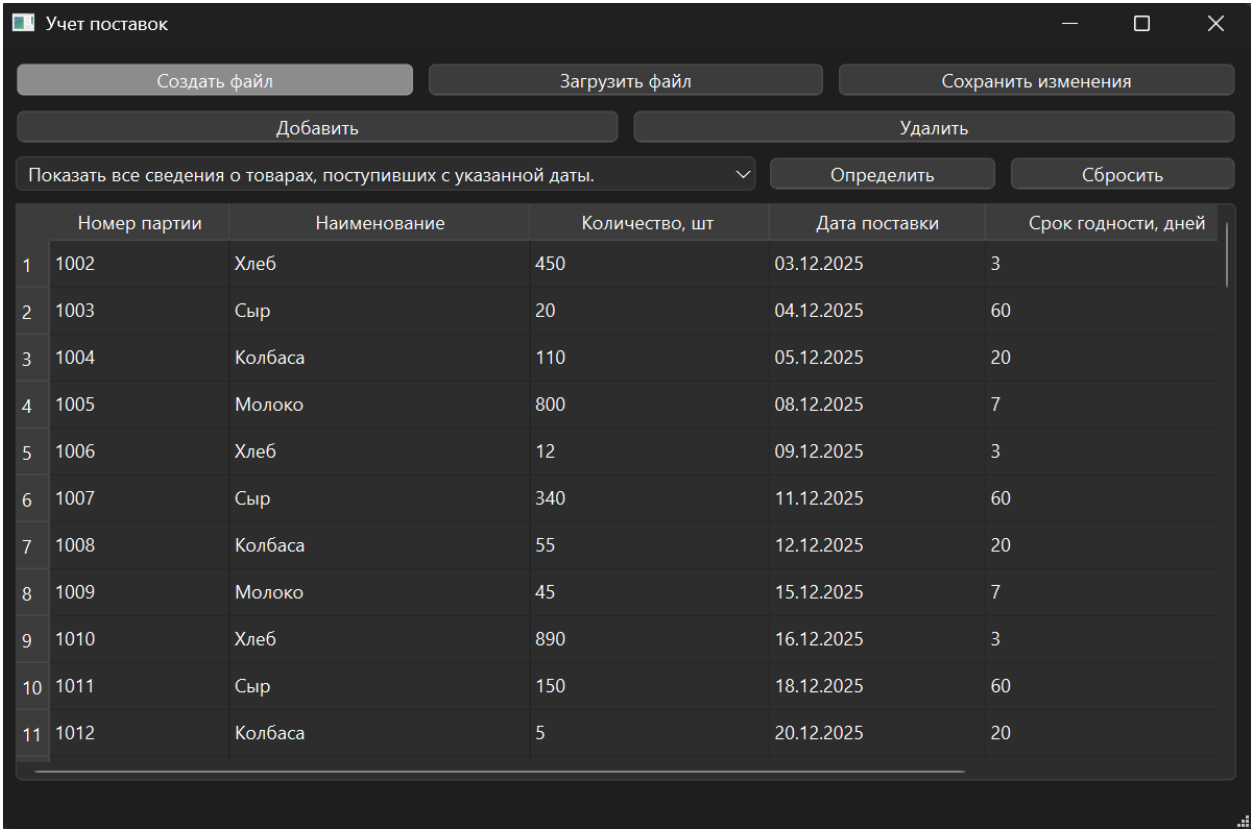
    chartView->setChart(newChart);
}

```

Рисунок 15 – Реализация chartForm

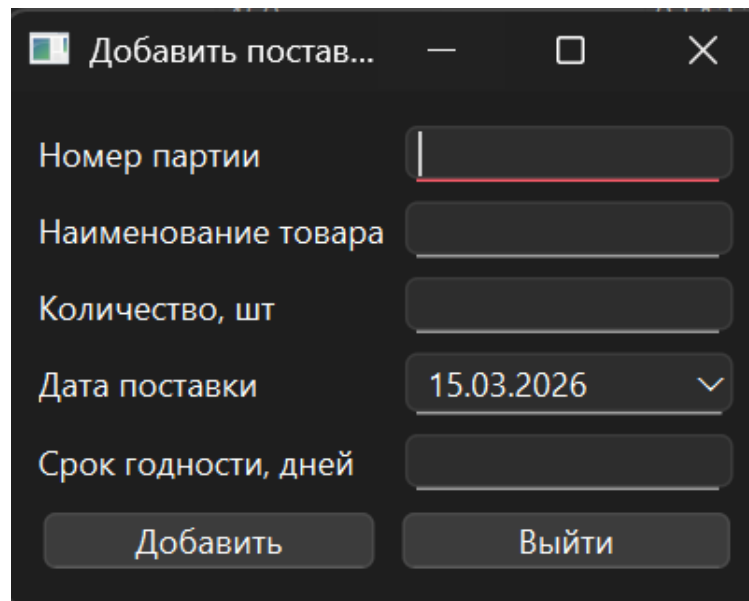
1.4 Тестирование приложения

После написания исходного кода было проведено тестирование программы. Результаты тестирования основных функций, включая добавление записей, отображение данных и построение графика поставок, представлены на скриншотах.



	Номер партии	Наименование	Количество, шт	Дата поставки	Срок годности, дней
1	1002	Хлеб	450	03.12.2025	3
2	1003	Сыр	20	04.12.2025	60
3	1004	Колбаса	110	05.12.2025	20
4	1005	Молоко	800	08.12.2025	7
5	1006	Хлеб	12	09.12.2025	3
6	1007	Сыр	340	11.12.2025	60
7	1008	Колбаса	55	12.12.2025	20
8	1009	Молоко	45	15.12.2025	7
9	1010	Хлеб	890	16.12.2025	3
10	1011	Сыр	150	18.12.2025	60
11	1012	Колбаса	5	20.12.2025	20

Рисунок 16 - Основное окно программы



Добавить постав...

Номер партии

Наименование товара

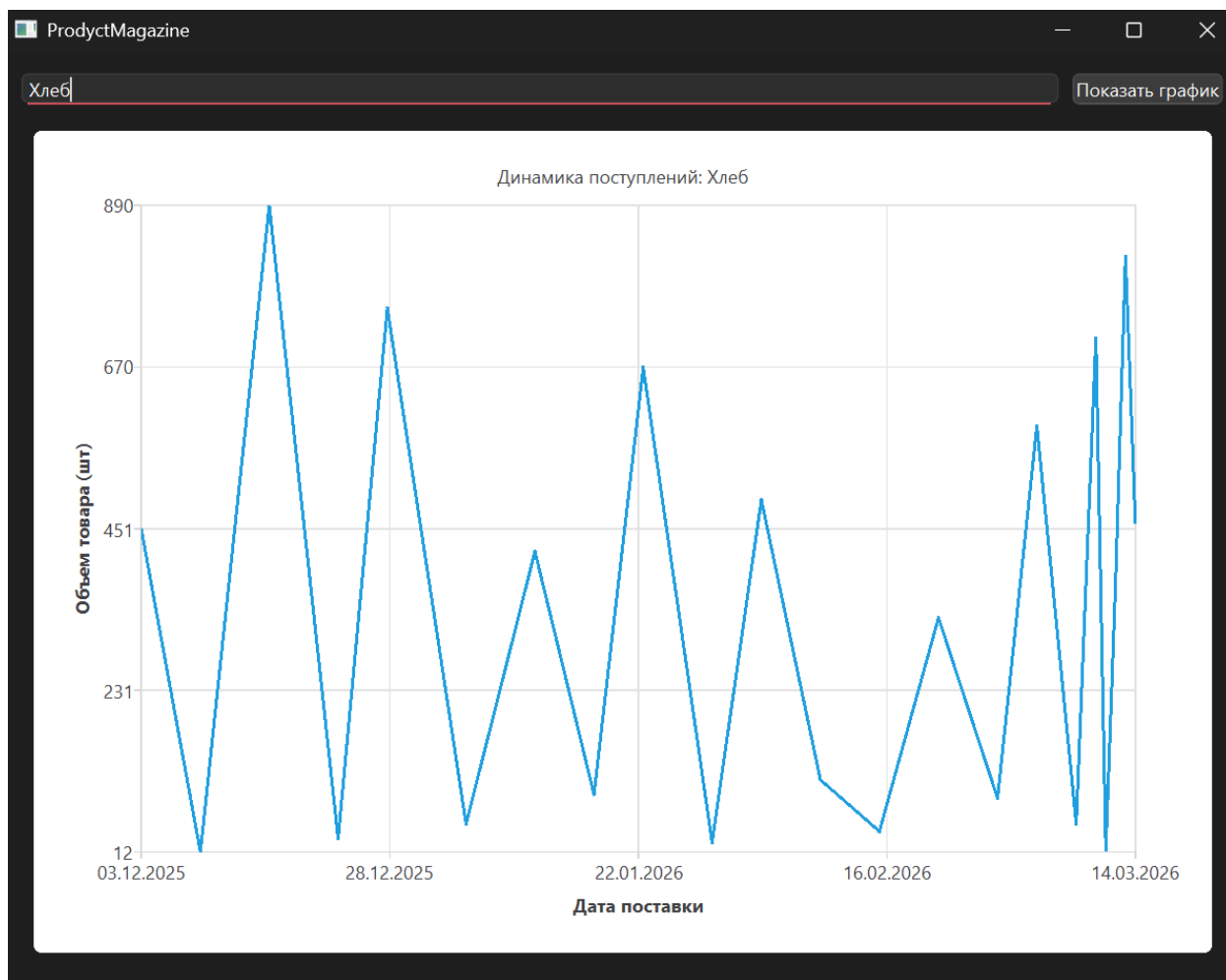
Количество, шт

Дата поставки 15.03.2026

Срок годности, дней

Добавить Выйти

Рисунок 17 - Диалоги создания



Задание 3. Создание приложения с графическим интерфейсом на языке C#.

Задание: Разработать приложение «Крестики-нолики». Интерфейс приложения будет состоять из:

- игрового поля: 9 клеток, в которых пользователями будут выставляться крестики (символ «X») и нолики (символ «O»);
- поля вывода результатов ходов, в которое будет выводиться номер клетки, в которую установил свой символ пользователь, и результаты игры;
- кнопки «Начать заново», перезапускающей игру.

Для упрощения программа будет разработана для двух игроков-людей. Она будет проверять, победил ли один из игроков после каждого хода, а в конце игры, когда все клетки будут заполнены, будет проверять, выиграл ли один из игроков или игра закончилась ничьей. После завершения игры либо победой, либо ничьей, программа не будет реагировать на любые действия пользователей до нажатия ими кнопки «Начать заново». По нажатию той же кнопки игра может быть завершена досрочно. Клетки игрового поля будут реализованы с помощью кнопок, по нажатию которых в названиях кнопок будут появляться символы «X» или «O».

3.1 Проектирование приложения

Для реализации игры была разработана диаграмма классов, описывающая структуру главного окна приложения и его компонентов.

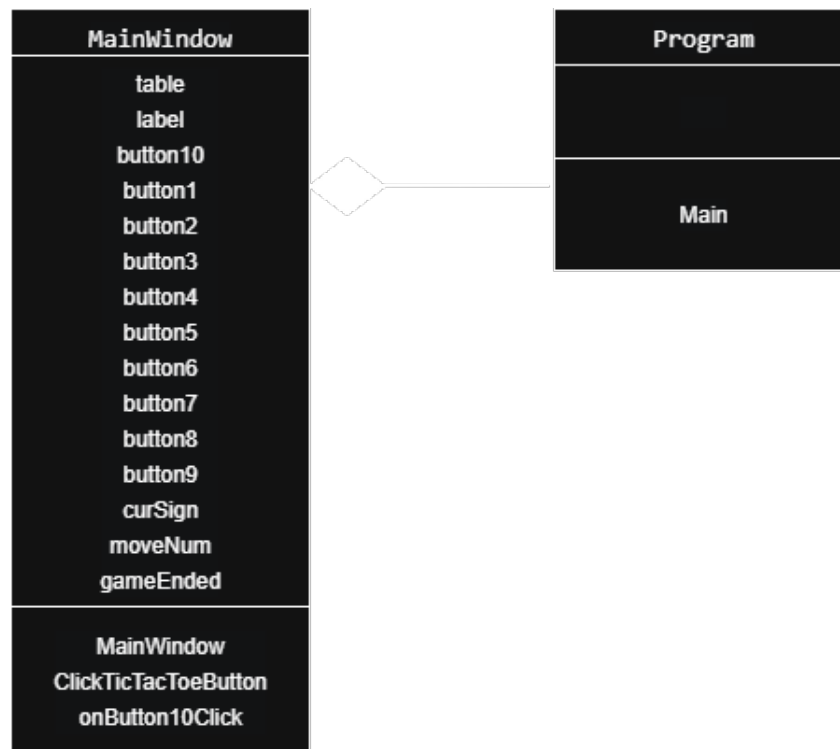


Рисунок 19 – Диаграмма классов

3.2 Разработка кода программы

На основе архитектуры был написан программный код, реализующий логику, проверку условий победы или ничьей, а также сброс состояния игры.

```
public partial class MainWindow : Gtk.Window
{
    String curSign;
    int moveNum;
    bool gameEnded;
    public MainWindow() : base(Gtk.WindowType.Toplevel)
    {
        Build();
        curSign = "X";
        moveNum = 1;
        gameEnded = false;
    }
    protected void OnButton10Clicked(object sender, EventArgs e)
    {
        curSign = "X";
        moveNum = 1;
        gameEnded = false;
        this.button1.Label = "";
        this.button2.Label = "";
        this.button3.Label = "";
        this.button4.Label = "";
        this.button5.Label = "";
        this.button6.Label = "";
        this.button7.Label = "";
        this.button8.Label = "";
        this.button9.Label = "";
        this.entry1.Text = "Игра началась!";
    }
}
```

Рисунок 20 – Код программы

```

private void ClickTicTacToeButton(object sender, EventArgs e)
{
    Button button = (Button)sender;
    if (string.IsNullOrEmpty(button.Label) & !gameEnded)
    {
        button.Label = curSign;
        if (!string.IsNullOrEmpty(this.button1.Label)
& this.button1.Label == this.button2.Label
& this.button2.Label == this.button3.Label
|
!string.IsNullOrEmpty(this.button4.Label)
& this.button4.Label == this.button5.Label
& this.button5.Label == this.button6.Label
|
!string.IsNullOrEmpty(this.button7.Label)
& this.button7.Label == this.button8.Label
& this.button8.Label == this.button9.Label
|
!string.IsNullOrEmpty(this.button1.Label)
& this.button1.Label == this.button4.Label
& this.button4.Label == this.button7.Label
|
!string.IsNullOrEmpty(this.button2.Label)
& this.button2.Label == this.button5.Label
& this.button5.Label == this.button8.Label
|
!string.IsNullOrEmpty(this.button3.Label)
& this.button3.Label == this.button6.Label
& this.button6.Label == this.button9.Label
|
!string.IsNullOrEmpty(this.button1.Label)
& this.button1.Label == this.button5.Label
& this.button5.Label == this.button9.Label
|
!string.IsNullOrEmpty(this.button3.Label)
& this.button3.Label == this.button5.Label
& this.button5.Label == this.button7.Label
)
        {
            gameEnded = true;
            this.entry1.Text = "Ход " + moveNum.ToString() +
": Победил " + curSign + "!";
        }
        else if (!string.IsNullOrEmpty(this.button1.Label) &
!string.IsNullOrEmpty(this.button2.Label) &
!string.IsNullOrEmpty(this.button3.Label) &
!string.IsNullOrEmpty(this.button4.Label) &
!string.IsNullOrEmpty(this.button5.Label) &
!string.IsNullOrEmpty(this.button6.Label) &
!string.IsNullOrEmpty(this.button7.Label) &
!string.IsNullOrEmpty(this.button8.Label) &
!string.IsNullOrEmpty(this.button9.Label)
)
        {
            gameEnded = true;
            this.entry1.Text = "Ход " + moveNum.ToString() +
": Ничья!";
        }
        else
        {
            this.entry1.Text = "Ход " + moveNum.ToString() +
": " + curSign + " сходил на " +
button.Name[6] + " клетку";
            if (curSign == "X")
                curSign = "O";
            else
                curSign = "X";
            moveNum += 1;
        }
    }
}
}

```

Рисунок 21 – Код программы

3.3 Тестирование программы

В ходе тестирования были проверены различные сценарии развития игры: победа первого игрока (крестики), победа второго игрока (нолики) и ничья. Программа корректно реагирует на действия пользователей и блокирует поле после завершения партии.

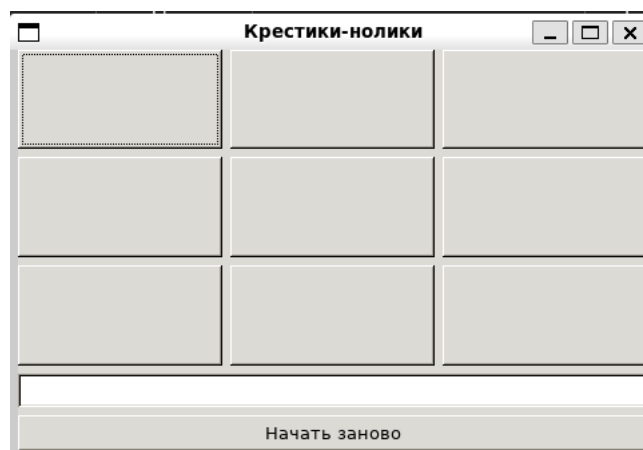


Рисунок 22 – Начало игры

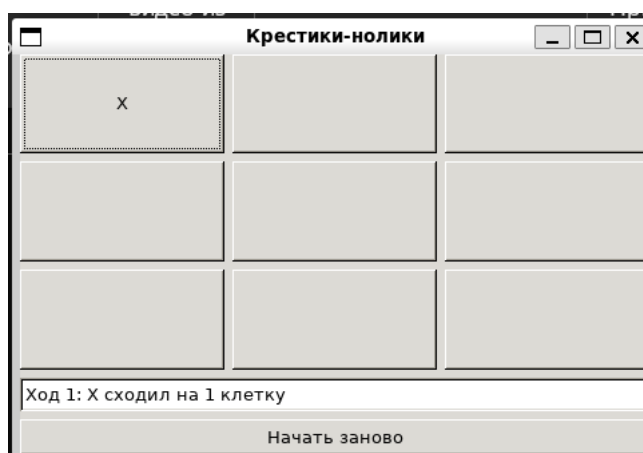


Рисунок 23 – Ход X

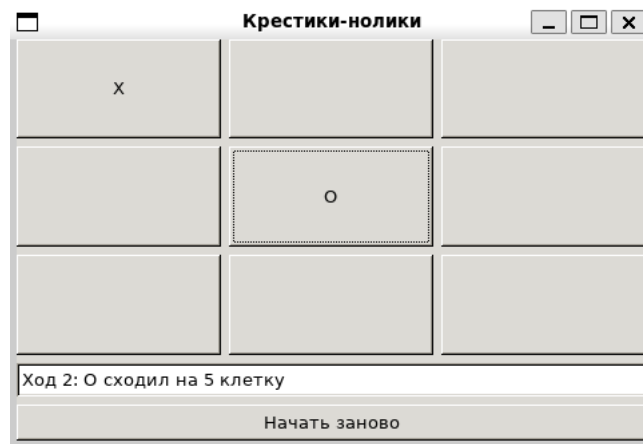


Рисунок 24 – Ход О



Рисунок 25 – Победа О



Рисунок 26 – Победа X

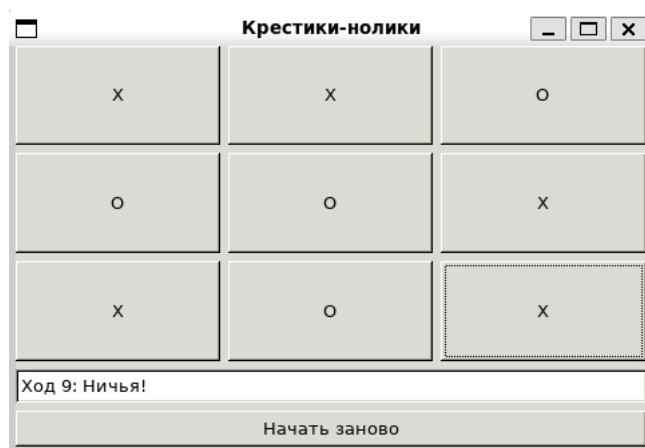


Рисунок 27 - Ничья

Вывод: в результате выполнения задания было разработано оконное приложение для игры в крестики-нолики. Реализована логика ходов двух игроков с автоматической проверкой победы или ничьей после каждого действия. Программа корректно блокирует кнопки после окончания партии и полностью сбрасывает состояние при начале новой игры. Проект позволил закрепить на практике навыки применения принципов объектно-ориентированного программирования и создания графических интерфейсов.

Задание 3 Создание программной системы с графическим интерфейсом на C#

Задание: Выполнить объектную декомпозицию, разработать формы интерфейса, диаграмму состояний интерфейса, диаграммы классов интерфейсной и предметной областей, диаграмму последовательности действий одной из реализуемых операций. Разработать, протестировать и отладить программу.

База данных (файл) продуктового магазина содержит сведения о товарах: номер партии, наименование товара, количество (штук), дата поставки, срок годности (в днях). Программа должна в интерактивном режиме формировать файл, добавлять и удалять данные, а также воспринимать каждый из перечисленных запросов и давать на него ответ.

1. Показать все сведения о товарах, поступивших с указанной даты.
2. Определить наименования товаров, срок годности которых истекает сегодня.
3. Определить самый скоропортящийся товар (если их несколько, показать все).

4.1 Проектирование приложения

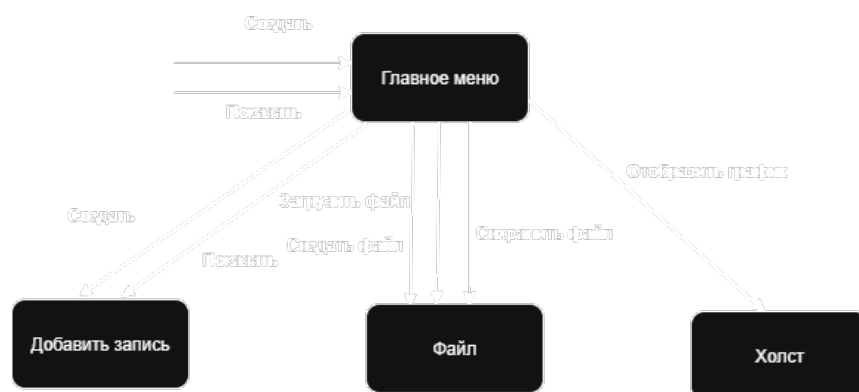


Рисунок 28 – Объектная декомпозиция

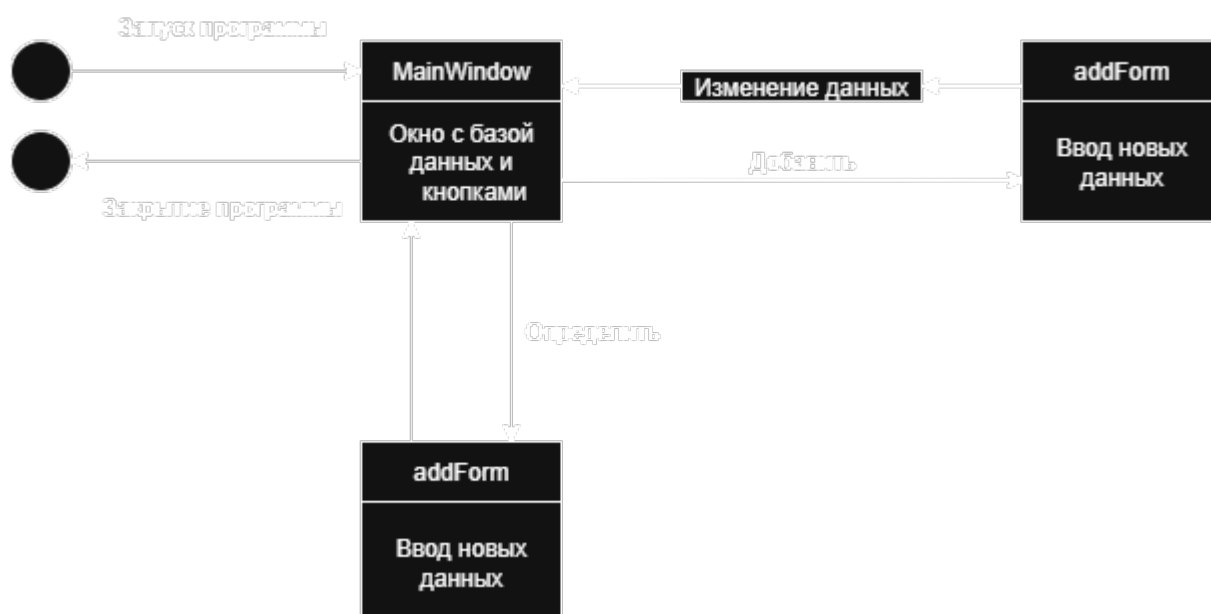


Рисунок 29 - Диаграмма классов интерфейса

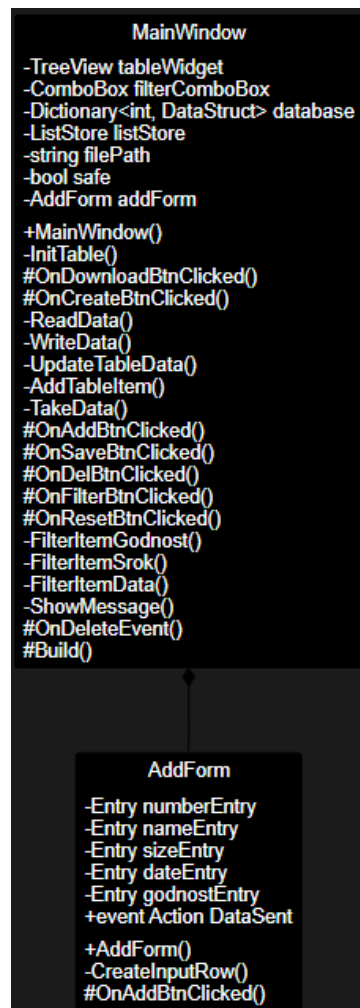


Рисунок 30 - Диаграмма классов интерфейсной области

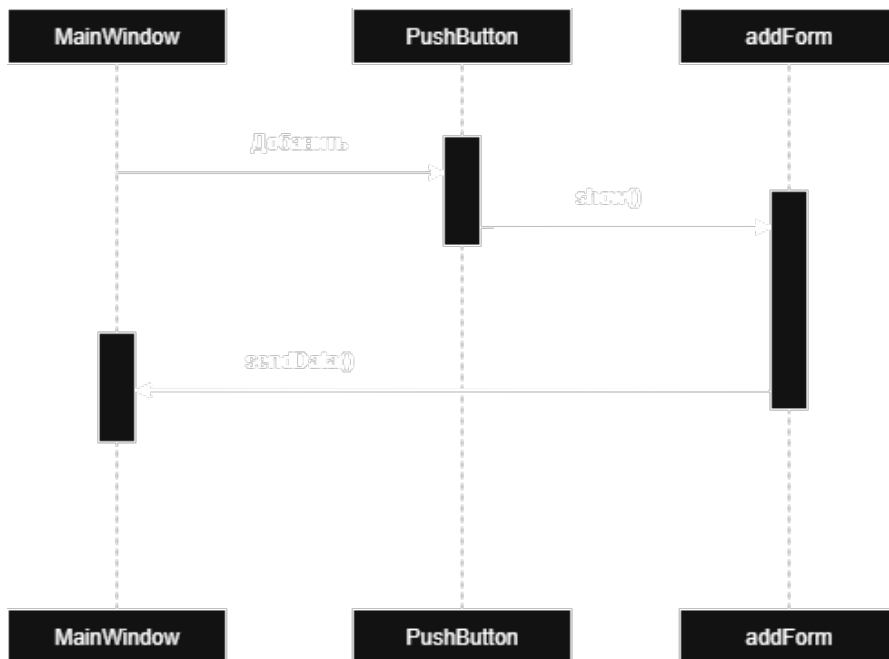


Рисунок 31 - Диаграмма последовательности действий

4.2 Разработка кода программы

```

public struct DataStruct
{
    public int Number;
    public string Name;
    public int Size;
    public string Date;
    public int Godnost;
}
  
```

Рисунок 32 – Структура для хранения данных

```

public partial class MainWindow : Gtk.Window
{
    private TreeView tableView;
    private ComboBox filterComboBox;

    private Dictionary<int, DataStruct> database = new Dictionary<int, DataStruct>();
    private ListStore listStore;
    private string filePath = "";
    private bool safe = true;

    private AddForm addForm;

    public MainWindow() : base(Gtk.WindowType.Toplevel)
    {
        this.Title = "Product Magazine";
        this.SetDefaultSize(800, 600);
        this.SetPosition(WindowPosition.Center);

        VBox mainVBox = new VBox(false, 2);

        HBox row1 = new HBox(true, 2);
        Button btnCreate = new Button("Создать файл");
        btnCreate.Clicked += OnCreateBtnClicked;
        Button btnLoad = new Button("Загрузить файл");
        btnLoad.Clicked += OnDownloadBtnClicked;
        Button btnSave = new Button("Сохранить изменения");
        btnSave.Clicked += OnSaveBtnClicked;

        row1.PackStart(btnCreate, true, true, 0);
        row1.PackStart(btnLoad, true, true, 0);
        row1.PackStart(btnSave, true, true, 0);
        mainVBox.PackStart(row1, false, false, 0);

        HBox row2 = new HBox(true, 2);
        Button btnAdd = new Button("Добавить");
        btnAdd.Clicked += OnAddBtnClicked;
        Button btnDel = new Button("Удалить");
        btnDel.Clicked += OnDelBtnClicked;

        row2.PackStart(btnAdd, true, true, 0);
        row2.PackStart(btnDel, true, true, 0);
        mainVBox.PackStart(row2, false, false, 0);

        HBox row3 = new HBox(false, 2);
        filterComboBox = ComboBox.NewText();
        filterComboBox.AppendText("Фильтр по дате");
        filterComboBox.AppendText("Срок истекает сегодня");
        filterComboBox.AppendText("Минимальная годность");
        filterComboBox.Active = 0; // Значение по умолчанию

        Button btnFilter = new Button("Определить");
        btnFilter.Clicked += OnFilterBtnClicked;
        Button btnReset = new Button("Сбросить");
        btnReset.Clicked += OnResetBtnClicked;

        row3.PackStart(filterComboBox, true, true, 0);

```

Рисунок 33 – Класс MainWindow

```

private void InitTable()
{
    listStore = new ListStore(typeof(int), typeof(string), typeof(int), typeof(string), typeof(int));
    tableView.Model = listStore;

    tableView.AppendColumn("Номер", new CellRendererText(), "text", 0);
    tableView.AppendColumn("Название", new CellRendererText(), "text", 1);
    tableView.AppendColumn("Объем", new CellRendererText(), "text", 2);
    tableView.AppendColumn("Дата поставки", new CellRendererText(), "text", 3);
    tableView.AppendColumn("Годность", new CellRendererText(), "text", 4);
}

protected void OnDownloadBtnClicked(object sender, EventArgs e)
{
    FileChooserDialog fileChooser = new FileChooserDialog(
        "Выберите файл для загрузки",
        this,
        FileChooserAction.Open,
        "Отмена", ResponseType.Cancel,
        "Открыть", ResponseType.Accept);

    FileFilter filter = new FileFilter();
    filter.Name = "Текстовые файлы (*.txt)";
    filter.AddPattern("*.txt");
    fileChooser.AddFilter(filter);

    if (fileChooser.Run() == (int)ResponseType.Accept)
    {
        filePath = fileChooser.Filename;
        ReadData();
    }
    fileChooser.Destroy();
}

protected void OnCreateBtnClicked(object sender, EventArgs e)
{
    FileChooserDialog fileChooser = new FileChooserDialog(
        "Создать и сохранить файл",
        this,
        FileChooserAction.Save,
        "Отмена", ResponseType.Cancel,
        "Сохранить", ResponseType.Accept);

    if (fileChooser.Run() == (int)ResponseType.Accept)
    {
        filePath = fileChooser.Filename;
        File.Create(filePath).Close();
        ReadData();
    }
    fileChooser.Destroy();
}

```

Рисунок 34 – Методы загрузки и создания файлов

```

private void ReadData()
{
    database.Clear();
    try
    {
        using (StreamReader sr = new StreamReader(filePath))
        {
            string line;
            while ((line = sr.ReadLine()) != null)
            {
                string[] parts = line.Split(';');
                if (parts.Length == 5)
                {
                    DataStruct dataTemp = new DataStruct
                    {
                        Number = int.Parse(parts[0]),
                        Name = parts[1],
                        Size = int.Parse(parts[2]),
                        Date = parts[3],
                        Godnost = int.Parse(parts[4])
                    };
                    database[dataTemp.Number] = dataTemp;
                }
            }
        }
        UpdateTableData();
    }
    catch (Exception ex)
    {
        ShowMessage(MessageType.Error, "Ошибка чтения файла: " + ex.Message);
    }
}

private void WriteData()
{
    try
    {
        using (StreamWriter sw = new StreamWriter(filePath))
        {
            foreach (var item in database)
            {
                sw.WriteLine($"{item.Key};{item.Value.Name};{item.Value.Size};{item.Value.Date};{item.Value.Godnost}");
            }
        }
    }
    catch (Exception ex)
    {
        ShowMessage(MessageType.Error, "Ошибка записи файла: " + ex.Message);
    }
}

```

Рисунок 35 – Методы чтения и записи данных в файл

```

protected void OnAddBtnClicked(object sender, EventArgs e)
{
    if (string.IsNullOrEmpty(filePath))
    {
        ShowMessage(MessageType.Warning, "Файл не выбран.");
        return;
    }
    addForm.ShowAll();
}

protected void OnSaveBtnClicked(object sender, EventArgs e)
{
    if (!string.IsNullOrEmpty(filePath))
    {
        WriteData();
        ShowMessage(MessageType.Info, "Данные сохранены.");
        safe = true;
    }
    else
    {
        ShowMessage(MessageType.Warning, "Файл не выбран.");
    }
}

protected void OnDelBtnClicked(object sender, EventArgs e)
{
    TreeSelection selection = tableWidget.Selection;
    if (selection.GetSelected(out TreeIter iter))
    {
        int id = (int)listStore.GetValue(iter, 0);
        database.Remove(id);
        UpdateTableData();
    }
    else
    {
        ShowMessage(MessageType.Warning, "Строка не выбрана.");
    }
}

protected void OnFilterBtnClicked(object sender, EventArgs e)
{
    int textIndex = filterComboBox.Active;
    switch (textIndex)
    {
        case 0: FilterItemData(); break;
        case 1: FilterItemSrok(); break;
        case 2: FilterItemGodnost(); break;
    }
}

protected void OnResetBtnClicked(object sender, EventArgs e)
{
    UpdateTableData();
}

```

Рисунок 36 – Методы добавления, удаления и фильтрации данных

```

private void FilterItemsrok()
{
    listStore.Clear();
    DateTime today = DateTime.Today;

    foreach (var kvp in database)
    {
        if (DateTime.TryParseExact(kvp.Value.Date, "dd.MM.yyyy", null, System.Globalization.DateTimeStyles.None, out DateTime parsedDate))
        {
            if (parsedDate.AddDays(kvp.Value.Godnost) == today)
            {
                AddTableItem(kvp.Key, kvp.Value);
            }
        }
    }
}

private void FilterItemData()
{
    MessageDialog md = new MessageDialog(this, DialogFlags.Modal, MessageType.Question, ButtonsType.OkCancel, "Введите дату в формате ДД.ММ.ГГГГ:");
    Entry entry = new Entry();
    md.VBox.Add(entry);
    md.ShowAll();

    if (md.Run() == (int)ResponseType.Ok && !string.IsNullOrEmpty(entry.Text))
    {
        if (DateTime.TryParseExact(entry.Text, "dd.MM.yyyy", null, System.Globalization.DateTimeStyles.None, out DateTime targetDate))
        {
            listStore.Clear();
            foreach (var kvp in database)
            {
                if (DateTime.TryParseExact(kvp.Value.Date, "dd.MM.yyyy", null, System.Globalization.DateTimeStyles.None, out DateTime itemDate))
                {
                    if (itemDate > targetDate)
                    {
                        AddTableItem(kvp.Key, kvp.Value);
                    }
                }
            }
        }
        else
        {
            ShowMessage(MessageType.Error, "Неверный формат даты!");
        }
    }
    md.Destroy();
}

```

Рисунок 37 – Методы фильтрации

```

public partial class AddForm : Gtk.Window
{
    private Entry numberEntry;
    private Entry nameEntry;
    private Entry sizeEntry;
    private Entry dateEntry;
    private Entry godnostEntry;

    public event Action<int, string, int, string, int> DataSent;

    public AddForm() : base(Gtk.WindowType.Toplevel)
    {
        this.Title = "Добавление товара";
        this.SetDefaultSize(350, 250);
        this.SetPosition(WindowPosition.Center);

        VBox vbox = new VBox(false, 5);
        vbox.BorderWidth = 10;

        numberEntry = CreateInputRow(vbox, "Номер:");
        nameEntry = CreateInputRow(vbox, "Название:");
        sizeEntry = CreateInputRow(vbox, "Объем:");
        dateEntry = CreateInputRow(vbox, "Дата (ДД.ММ.ГГГГ):");
        godnostEntry = CreateInputRow(vbox, "Годность:");

        Button btnAdd = new Button("Подтвердить");
        btnAdd.Clicked += OnAddBtnClicked;

        vbox.PackStart(btnAdd, false, false, 10);

        this.Add(vbox);
    }

    private Entry CreateInputRow(VBox parent, string labelText)
    {
        HBox hbox = new HBox(false, 5);
        Label label = new Label(labelText);
        label.WidthRequest = 140;
        label.Xalign = 0;

        Entry entry = new Entry();
        hbox.PackStart(label, false, false, 0);
        hbox.PackStart(entry, true, true, 0);

        parent.PackStart(hbox, false, false, 0);
        return entry;
    }
}

```

Рисунок 38 – Класс окна добавления нового элемента

```

private Entry CreateInputRow(VBox parent, string labelText)
{
    HBox hbox = new HBox(false, 5);
    Label label = new Label(labelText);
    label.WidthRequest = 140;
    label.Xalign = 0;

    Entry entry = new Entry();
    hbox.PackStart(label, false, false, 0);
    hbox.PackStart(entry, true, true, 0);

    parent.PackStart(hbox, false, false, 0);
    return entry;
}

protected void OnAddBtnClicked(object sender, EventArgs e)
{
    try
    {
        int number = int.Parse(numberEntry.Text);
        string name = nameEntry.Text;
        int size = int.Parse(sizeEntry.Text);
        string date = dateEntry.Text;
        int godnost = int.Parse(godnostEntry.Text);

        DataSent?.Invoke(number, name, size, date, godnost);

        numberEntry.Text = "";
        nameEntry.Text = "";
        sizeEntry.Text = "";
        dateEntry.Text = "";
        godnostEntry.Text = "";

        this.Hide();
    }
    catch (Exception)
    {
        MessageDialog md = new MessageDialog(this, DialogFlags.Modal, MessageType.Error, ButtonsType.Ok, "Ошибка ввода данных: проверьте форматы чисел.");
        md.Run();
        md.Destroy();
    }
}

```

Рисунок 39 – Реализация кнопки добавления

Вывод: В процессе разработки выполнена объектная декомпозиция, написан графический интерфейс, добавлены функции работы с файлами, фильтрации записей и вывода статистики. Готовая программа протестирована и полностью соответствует условиям задачи.